SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance (BL3-BL4)

Assessment Tool Weightage: 30%

Dr.J.Velmurugan / Professor
Dept. of CSE

QUESTIONS: 10 **Total Marks:50**

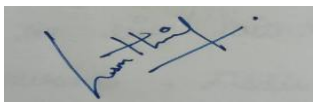
Annexure A

Debugging or Optimization task

Code of Conduct:

I _Hasmath nisha (192524513) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Criteria	Weightage (Marks)	Level 4 – Exemplary (Full Marks)	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Total
Problem Identification	**20%**					
Debugging	**25%**					
Optimization	**20%**					
Analysis	**20%**					

Code Quality	**15%**					
Total						

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)

Process P2:
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)

Identify the exact condition that causes deadlock.
Suggest an optimisation to eliminate the deadlock without changing process logic.

(10 Marks)

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.
You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(10 Marks)

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB

- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(10 Marks)

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(10 Marks)

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.
Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(10 Marks)

Operating Systems: Debugging and Optimisation

Q1. Deadlock Debugging (Code Trace)

> *Two processes share R1 and R2. P1 locks R1, waits 100 ms, then locks R2. P2 locks R2, waits 100 ms, then locks R1. Identify the exact condition causing deadlock and suggest an optimisation without changing process logic.*

Trace of execution

Time	P1	P2
t = 0	lock(R1) succeeds	lock(R2) succeeds
t = 0 to 100 ms	wait(100 ms) while holding R1	wait(100 ms) while holding R2
t = 100 ms	lock(R2) blocks, R2 is held by P2	lock(R1) blocks, R1 is held by P1
After	Waits for P2 forever	Waits for P1 forever

Exact condition causing deadlock

The two processes acquire the same two locks in **opposite order** (P1: R1 then R2; P2: R2 then R1). The 100 ms wait widens the window so that each process grabs its first lock before the other asks for it. This creates a **circular wait**: P1 holds R1 and waits for R2, while P2 holds R2 and waits for R1.

All four Coffman conditions hold at once:

- **Mutual exclusion:** a lock can be held by only one process.
- **Hold and wait:** each process holds one lock while requesting another.
- **No pre-emption:** a lock cannot be forcibly taken away.
- **Circular wait:** P1 → R2 → P2 → R1 → P1 forms a cycle in the resource allocation graph.

Breaking any one condition removes the deadlock; the cheapest to break is circular wait.

Optimisation (without changing process logic)

Impose a global lock ordering. Give every resource a fixed rank (R1 < R2) and require every process to request locks in ascending rank. P2 then takes R1 first and R2 second. The critical section and the wait are unchanged; only the acquisition order of the lock calls is standardised.

```
Process P2 (after ordering):
  lock(R1)      // lower rank first
  wait(100 ms)
  lock(R2)
  critical section
  unlock(R2)
  unlock(R1)
```

Why it works: whoever gets R1 first can always proceed to R2, because nobody holding R2 is ever waiting for R1. The wait-for graph can never contain a cycle.

Other valid options if reordering is not allowed:

- **Timed try-lock with back-off:** if lock(R2) fails within a timeout, release R1 and retry after a random delay (breaks hold and wait and no pre-emption).
- **Acquire all resources atomically:** take both locks together or neither (breaks hold and wait).
- **Deadlock detection and recovery:** detect cycles in the wait-for graph and abort one process.

Q2. CPU Utilisation Problem

> *CPU utilisation is only 30% even though processes are ready. The scheduler log shows frequent transitions: ready → waiting → ready → waiting → ready ...*

Root cause

The processes are **I/O-bound**. Each one runs for a very short CPU burst, issues an I/O request, and moves to the waiting (blocked) state. When the I/O completes it returns to ready, runs briefly, and blocks again. Because the bursts are tiny and I/O waits are long, there are moments when **every process is blocked on I/O** and the CPU sits idle. Overheads from the constant state changes (context switches, interrupt handling) make it worse.

Contributing factors:

- Low degree of multiprogramming: too few processes in memory, so there is nothing to run while others wait on I/O.
- Slow or contended I/O devices (disk or network), giving long waiting times.
- Synchronous (blocking) I/O calls, so a process cannot overlap computation with I/O.
- Scheduler that does not favour processes whose I/O has just finished, so the CPU stays idle or runs poorly-matched work.

System-level optimisations

1. **Increase the degree of multiprogramming.** Admit more processes into memory so the CPU always has a ready process while others wait on I/O. Monitor so it does not cause thrashing.
2. **Mix CPU-bound and I/O-bound jobs.** The long-term (job) scheduler should balance the job mix so the CPU and I/O devices are both kept busy.
3. **Use asynchronous / non-blocking I/O and buffering or spooling.** Let a process continue computing while I/O is in progress; use DMA, read-ahead and caching to cut wait times.
4. **Use a multilevel feedback queue (MLFQ).** Give I/O-bound processes higher priority so they get the CPU immediately after I/O completes, issue their next request sooner, and keep devices busy in parallel with CPU work.
5. **Use faster or parallel I/O.** Faster disks or SSDs, multiple I/O channels, and better disk scheduling reduce the time spent in the waiting state.

Expected result: the waiting and ready periods overlap across many processes, so the CPU is idle far less often and utilisation rises well above 30%.

Q3. Memory Bottleneck Debugging

> *A multi-threaded program has frequent page faults. Memory map: Code 5 MB, Heap 150 MB, Stack 1 MB per thread, 25 threads, physical RAM available 2 GB. Identify the issue and propose two optimisations.*

Memory footprint calculation

Component	Size	Calculation
Code	5 MB	shared by all threads
Heap	150 MB	shared by all threads
Stacks	25 MB	25 threads × 1 MB
Total virtual footprint	180 MB	5 + 150 + 25
Physical RAM available	2048 MB	2 GB

Diagnosis

The total footprint (about 180 MB) is only about **9% of RAM**, so the problem is **not a shortage of physical memory** and not thrashing caused by over-commitment. The page faults are caused by **poor locality of reference and inefficient memory access** within the 150 MB shared heap:

- Threads access scattered heap data (poor spatial and temporal locality), so the working set keeps shifting and pages are repeatedly faulted in.
- Heap fragmentation and many small allocations spread data across many pages.
- Cold-start (demand paging) faults: pages are loaded lazily on first access, and 25 threads touching different regions cause many first-touch faults.
- Possible false sharing and cache-line contention between threads on shared structures, which hurts cache and TLB behaviour and adds to the overhead.
- If pages were swapped or the memory is not locked, the OS may reclaim pages that are still in use.

So the bottleneck is an inefficient access pattern and page management policy, not capacity.

Optimisation 1: Improve locality and pre-fault memory

Restructure data so each thread works on a contiguous region (thread-local arenas, array-of-structures to structure-of-arrays, cache-friendly traversal). Pre-fault or pre-load the hot heap at start-up (for example touch pages or use MAP_POPULATE / madvise WILLNEED) and lock the working set in RAM (mlock) so it is not paged out. This cuts both first-touch faults and repeated faults.

Optimisation 2: Use huge pages and a better allocator

Back the 150 MB heap with large pages (2 MB) to reduce the number of pages from about 38,400 (4 KB pages) to about 75, which sharply reduces page faults and TLB misses. Combine with a thread-caching allocator (such as jemalloc or tcmalloc) to reduce fragmentation and cross-thread contention.

Additional options: tune the page replacement policy / working-set size, and pad shared data to cache-line boundaries to avoid false sharing.

Q4. Starvation Debugging

> *In a multilevel queue scheduler, P5 (in Q3, Batch) never gets CPU time. Q1 = Interactive (highest, 5 ms quantum), Q2 = System (10 ms quantum), Q3 = Batch (FCFS). Debug the root cause of starvation and suggest an optimisation for fairness.*

Root cause

A multilevel queue scheduler uses **strict (fixed) priority between queues**: the CPU serves Q3 only when Q1 and Q2 are completely empty. If interactive and system jobs keep arriving (very common on a busy system), Q1 and Q2 are never empty, so Q3 is never reached. P5 is stuck behind them.

Two further effects make it worse:

- Processes are permanently assigned to a queue and cannot move between queues, so P5 has no way to rise in priority however long it waits.
- Q3 uses FCFS, so even when Q3 gets the CPU, P5 must wait for every job ahead of it to finish, and a long batch job can hold the CPU.

Conclusion: this is indefinite blocking (starvation) caused by strict priority with no aging and no inter-queue time sharing.

Optimisation for fairness

6. **Aging (primary fix).** Gradually raise the priority of a process the longer it waits. For example, if P5 waits more than a threshold T (say 100 ms or 1 second), promote it to Q2 and then Q1. After it gets service, its priority resets. This guarantees P5 eventually runs.
7. **Time-slice between queues.** Instead of strict priority, divide CPU time among the queues, for example 60% to Q1, 30% to Q2, 10% to Q3. Q3 is then guaranteed a minimum share even when higher queues are busy.
8. **Convert to a multilevel feedback queue (MLFQ).** Let processes move between queues, with periodic priority boosts (moving all jobs back to the top queue every S seconds) so no job is starved.
9. **Use round robin inside Q3** instead of FCFS, so a long batch job cannot block P5 once the queue is served.

Result: P5 is guaranteed a bounded waiting time, and interactive jobs still get good response because the higher-priority queues keep most of the CPU share.

Q5. Context-Switch Overhead Debugging

> *A server performs 20,000 context switches/sec. Average burst time per thread is 2 ms, I/O wait frequency is very high, and the preemptive scheduler uses a quantum of 1 ms. Explain why context switching is so high and propose two optimisations.*

Why context switching is so high

1. The quantum (1 ms) is smaller than the average CPU burst (2 ms). A thread needing 2 ms of CPU is pre-empted after 1 ms, put back in the ready queue, and must be switched in again to finish. Each burst therefore needs at least 2 context switches (and more with queueing), purely because of the timer, with no useful work gained.

2. Very frequent I/O waits. Threads block voluntarily on I/O before using up their quantum. Every block forces a switch to another thread, and every I/O completion makes a thread ready again and often triggers another switch.

3. Many short-lived runnable threads. With many threads each doing tiny bursts, the scheduler is invoked continually, so involuntary (timer-based) switches and voluntary (I/O-based) switches add together.

Quick estimate: with 20,000 switches/sec, the interval between switches is 1 s / 20,000 = 50 microseconds on average. If each switch costs about 5 to 10 microseconds (register save/restore, scheduler code, cache and TLB pollution), then 10% to 20% or more of the CPU time is lost to overhead before counting the cost of cold caches after each switch.

Optimisation 1: Increase the time quantum

Set the quantum above the typical burst, for example 5 to 10 ms (larger than the 2 ms average burst), so most threads finish their burst without being pre-empted. This removes the extra timer-driven switches. The quantum should remain small enough to keep response time acceptable, so a common rule is that about 80% of bursts should finish within one quantum.

Optimisation 2: Use a thread pool and asynchronous / event-driven I/O

Replace thread-per-request blocking I/O with a fixed pool of worker threads (about the number of cores) and non-blocking I/O (epoll, kqueue, io_uring, or an event loop). Threads no longer block and wake constantly, so far fewer switches occur. Where possible use user-level (green) threads or coroutines, whose switches are much cheaper than kernel context switches.

Additional improvements:

- Batch I/O completions and use interrupt coalescing to reduce wakeups.
- Use CPU affinity (pin threads to cores) to keep caches and TLBs warm after a switch.
- Use an adaptive scheduler that lengthens the quantum for CPU-bound threads and favours I/O-bound ones.

Rubrics for Calculation - Debugging or Optimization task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation